

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN MƯỜI BA
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Sau khi đã bổ sung các chức năng quản trị vận hành quan trọng như Audit Log, Camera Management và Demo Mode trong tuần thứ mười hai, tuần thứ mười ba của dự án tập trung vào việc ổn định hệ thống, kiểm thử các luồng tích hợp và xử lý các vấn đề phát sinh trong quá trình vận hành thực tế.

Ở giai đoạn trước, hệ thống đã có đầy đủ các thành phần chính như AI realtime detection, quản lý camera, kiểm duyệt vi phạm, lưu bằng chứng, thống kê, audit log và demo video. Tuy nhiên, khi các thành phần này được kết hợp với nhau, hệ thống phát sinh thêm một số vấn đề liên quan đến hiệu năng RTSP camera, quản lý phiên đăng nhập, xử lý token hết hạn, chuẩn hóa datetime và đồng bộ lỗi giữa Backend và Frontend.

Vì vậy, mục tiêu chính trong tuần này không phải là mở thêm nhiều tính năng lớn mới, mà là hoàn thiện và làm chắc các chức năng hiện có. Trọng tâm của tuần thứ mười ba bao gồm:

- Ổn định luồng camera RTSP lên hệ thống Live Monitoring
- Phân tích và xử lý nguyên nhân camera FPS thấp khi dùng IP Webcam
- Cải thiện phân quyền truy cập theo trạng thái của tài khoản
- Sửa lỗi frontend tự logout khi người dùng nhập sai mật khẩu liên tục
- Bổ sung refresh token để phiên làm việc không bị gián đoạn khi access token hết hạn
- Chuẩn hóa cách xử lý lỗi xác thực trên cả Backend và Frontend
- Chuẩn hóa datetime/timzone để tránh lỗi so sánh datetime trong SQLite
- Dọn lại .env.example và README để hướng dẫn cấu hình, cài đặt và vận hành hệ thống

Các công việc này giúp hệ thống trở nên ổn định hơn, dễ sử dụng hơn và gần với cách vận hành thực tế hơn. Đặc biệt, việc bổ sung refresh token và chuẩn hóa luồng xác thực giúp cải thiện trải nghiệm người dùng khi sử dụng các trang cần mở lâu như Live Monitoring, Violation History và Analytics.

2. Ổn định RTSP Camera

2.1. Vấn đề gặp phải

Trong quá trình kiểm thử với IP Webcam trên điện thoại, em phát hiện luồng camera khi xem trực tiếp trên trình duyệt của ứng dụng IP Webcam khá mượt, nhưng khi đưa vào hệ thống Live Monitoring thì chỉ đạt khoảng 6-7 FPS. Trong khi đó, AI FPS vẫn đạt khoảng 19-21 FPS.

Ban đầu có thể nhầm rằng vấn đề nằm ở mô hình YOLO hoặc inference pipeline. Tuy nhiên, khi phân tích kỹ telemetry, có thể thấy:

- Capture FPS thấp
- AI FPS vẫn cao

Điều này cho thấy bottleneck không nằm ở mô hình AI, mà nằm ở đoạn Backend đọc frame từ camera điện thoại thông qua OpenCV/FFmpeg.

2.2. Phân tích nguyên nhân

Nguyên nhân chính được xác định là cơ chế bỏ qua frame cũ trong RTSP capture loop. Trước đó, với RTSP stream, hệ thống có đoạn xử lý bỏ qua các frame cũ trong buffer để lấy frame mới hơn, từ đó giảm độ trễ khi xem camera RTSP. Tuy nhiên, khi kiểm thử thực tế với camera điện thoại, cơ chế này lại làm giảm mạnh Capture FPS.

Em đã viết script test độc lập để đo tốc độ đọc camera bằng OpenCV. Kết quả cho thấy:

- Không grab skip: khoảng 31.3 FPS
- Grab skip 5 frame: khoảng 5.3 FPS

Kết quả này chứng minh rằng bản thân camera điện thoại và OpenCV có thể đọc stream ở mức khoảng 30 FPS. Việc FPS tụt xuống khoảng 5 FPS chủ yếu là do mỗi vòng capture đang bỏ qua 5 frame rồi mới đọc 1 frame. Về mặt tỷ lệ, kết quả này gần tương ứng với $31 / 6$, tức là cứ 6 frame thì chỉ lấy 1 frame để xử lý.

2.3. Giải pháp thực hiện

Sau khi xác định nguyên nhân, em đã điều chỉnh lại cách xử lý RTSP capture.

Các thay đổi chính bao gồm:

- Thử nghiệm các mức skip frame khác nhau như 0, 1 và 3
- Kiểm tra lại tác động của skip frame đến FPS và độ trễ
- Cuối cùng lựa chọn cách loại bỏ cơ chế grab(5) khỏi pipeline

Bên cạnh đó, em cũng bổ sung và chuẩn hóa các biến cấu hình liên quan đến RTSP, ví dụ:

- RTSP_TRANSPORT=tcp
- RTSP_CAPTURE_BUFFER_SIZE=1

Việc đưa các tham số này vào cấu hình giúp hệ thống linh hoạt hơn khi chạy ở các môi trường mạng khác nhau. Trong một số trường hợp, udp có thể giảm độ trễ nếu Wi-Fi ổn định, nhưng tcp lại thường ổn định hơn khi mạng có nhiễu hoặc mất gói.

2.4. Kết quả đạt được

Sau khi loại bỏ cơ chế skip 5 frame, Capture FPS của camera điện thoại tăng trở lại khoảng 30 FPS. Đây là cải thiện rất rõ rệt so với mức 6-7 FPS trước đó.

Tuy nhiên, độ trễ hiển thị vẫn còn khoảng 1 giây. Độ trễ này được xác định là hợp lý với kiến trúc hiện tại, vì hệ thống không hiển thị trực tiếp luồng camera raw từ điện thoại. Luồng video phải đi qua nhiều bước:

- Camera điện thoại encode RTSP/H264
- Truyền qua Wi-Fi
- Backend dùng OpenCV/FFmpeg decode frame
- AI inference và tracking
- Vẽ bounding box
- Encode lại thành JPEG/MJPEG
- FastAPI stream về Frontend
- Browser hiển thị

Do đó, hệ thống có thêm độ trễ tự nhiên so với trang xem camera trực tiếp của IP Webcam. Tuy nhiên, với Capture FPS khoảng 30 FPS và AI FPS khoảng 18-21 FPS, hệ thống đã đạt mức ổn định đủ tốt cho bài toán realtime helmet detection.

3. Cải thiện trang Violation History

3.1. Bổ sung filter theo camera

Sau khi hệ thống có Camera Management, Violation History cần hỗ trợ lọc dữ liệu theo camera. Trước đó, hệ thống đã có dữ liệu camera metadata trong violation, nhưng giao diện filter vẫn chưa tận dụng đầy đủ.

Trong tuần này, em đã thêm filter theo camera. Backend /violations và /violations/export được bổ sung query camera_code, còn Frontend gọi API với tham số này.

Dropdown camera trên Frontend lấy dữ liệu từ API camera sources và hiển thị theo dạng: CAM_CODE - Camera Name

Điều này giúp người dùng dễ dàng lọc các vi phạm theo từng camera cụ thể, ví dụ camera cổng chính, camera bãi xe hoặc camera demo.

3.2. Hiển thị metadata camera trong violation detail

Trong modal xem ảnh bằng chứng, hệ thống bổ sung thêm thông tin camera metadata, bao gồm:

- Camera code
- Camera name
- Camera location
- Camera source type
- Demo badge nếu là dữ liệu demo

Việc hiển thị các thông tin này giúp người dùng hiểu rõ vi phạm được phát hiện ở đâu, từ camera nào và dữ liệu đó là dữ liệu thật hay dữ liệu demo.

Đồng thời, để giao diện bảng không quá chật, camera badge được bỏ khỏi từng hàng trong Violation Table, nhưng badge Demo vẫn được giữ lại vì đây là thông tin quan trọng để phân biệt dữ liệu demo và dữ liệu thật.

4. Cải thiện quản lý truy cập

4.1. Rà soát lại dependency phân quyền

Trong tuần này, em đã rà soát lại các dependency phân quyền ở Backend. Hệ thống hiện phân tách rõ các mức:

- CurrentUser
- ActiveUser
- VerifiedUser
- allow_admin
- allow_any_staff

Trong đó:

- `CurrentUser` chỉ yêu cầu token hợp lệ
- `ActiveUser` yêu cầu user active
- `VerifiedUser` yêu cầu user active và đã xác thực email
- `allow_admin` và `allow_any_staff` dựa trên `VerifiedUser`

Việc điều chỉnh này giúp các chức năng nghiệp vụ quan trọng như Live Monitoring, Violations, Analytics, Camera Management và Audit Logs chỉ được truy cập bởi người dùng đã active và verified.

4.2. Xử lý user chưa xác thực email

Trước đó, nếu user chưa xác thực email nhưng cố truy cập các chức năng nghiệp vụ, trải nghiệm lỗi chưa rõ ràng. Trong tuần này, Backend được bổ sung handler riêng cho lỗi `UserNotVerified`.

Frontend cũng được cập nhật để nhận diện lỗi `USER_NOT_VERIFIED` và hiển thị thông báo: “Please verify your email before using this feature.”

Ngoài ra, Sidebar cũng chặn click vào các chức năng nghiệp vụ nếu user chưa verify. Nếu user truy cập trực tiếp vào các route như `/live`, `/violations`, `/analytics`, `/users`, `/cameras`, `/audit-logs` khi chưa verify, hệ thống sẽ redirect về trang Profile.

Quy tắc hiện tại là:

- User chưa verify chỉ được dùng Profile, verify flow và logout
- User đã verify được dùng chức năng theo role
- Admin-only gồm User Management, Camera Management và Audit Logs

Cách xử lý này giúp hệ thống nhất quán hơn giữa Backend và Frontend.

5. Bổ sung Refresh Token

5.1. Lý do bổ sung refresh token

Trước tuần này, hệ thống chủ yếu dựa vào access token. Khi access token hết hạn, người dùng phải đăng nhập lại. Với hệ thống giám sát camera realtime, việc phiên đăng nhập bị ngắt sau một khoảng thời gian ngắn có thể gây khó chịu, đặc biệt khi người dùng đang theo dõi Live Monitoring hoặc kiểm tra dữ liệu vi phạm.

Vì vậy, trong tuần này em đã bổ sung cơ chế refresh token để giúp phiên làm việc mượt hơn. Access token vẫn có thời gian sống ngắn để đảm bảo an toàn, còn refresh token được dùng để cấp lại access token khi cần.

5.2. Thiết kế refresh token backend

Thay vì dùng refresh token dạng JWT stateless, hệ thống sử dụng refresh token dạng opaque random token.

Đặc điểm chính:

- Refresh token là chuỗi random
- Database chỉ lưu hash SHA-256 của refresh token
- Không lưu plain refresh token trong DB
- Refresh token được lưu trong HttpOnly cookie ở phía client

Em đã bổ sung model mới: RefreshToken

Các trường chính bao gồm:

- user_id
- token_hash
- expires_at
- revoked_at
- replaced_by
- user_agent
- ip_address
- last_used_at
- created_at

Cách thiết kế này giúp Backend có thể quản lý, revoke và rotate refresh token.

Nếu token bị logout hoặc bị thay thế, hệ thống có thể đánh dấu revoked thay vì chỉ chờ token hết hạn.

5.3. Luồng login, refresh và logout

Luồng login hiện tại:

- Người dùng nhập username/password
- Backend xác thực user
- Tạo access token
- Tạo refresh token record trong DB
- Set refresh token vào HttpOnly cookie
- Trả access token về Frontend

Luồng refresh:

- Frontend gọi /auth/refresh khi access token hết hạn
- Backend lấy refresh token từ cookie
- Hash token và tìm record trong DB
- Kiểm tra token tồn tại, chưa hết hạn, chưa revoked
- Kiểm tra user còn active
- Tạo access token mới
- Rotate refresh token cũ sang token mới
- Set refresh token mới vào cookie

Luồng logout:

- Frontend gọi /auth/logout
- Backend lấy refresh token từ cookie
- Revoke refresh token nếu tồn tại
- Clear cookie
- Frontend clear local session

Việc dùng HttpOnly cookie giúp Frontend không cần đọc trực tiếp refresh token, giảm rủi ro token bị truy cập bởi JavaScript.

5.4. Refresh token rotation

Hệ thống đã bổ sung cơ chế rotate refresh token. Khi refresh thành công, refresh token cũ bị revoke và token mới được tạo ra. Điều này giúp giảm rủi ro nếu refresh token cũ bị lộ.

Trong quá trình review, em cũng xác định cần đảm bảo rotation là thao tác atomic để tránh race condition khi nhiều tab hoặc nhiều request refresh cùng lúc. Vì vậy, logic rotation được định hướng sử dụng update có điều kiện:

```
WHERE revoked_at IS NULL
```

```
AND expires_at > now
```

Sau đó kiểm tra rowcount. Nếu rowcount != 1, nghĩa là token đã bị request khác revoke trước đó hoặc đã hết hạn, hệ thống không tạo token mới.

Đây là hướng xử lý an toàn hơn so với việc tách riêng “check valid” và “revoke” thành hai bước độc lập.

5.5. Refresh token frontend

Ở Frontend, api.js được cập nhật để hỗ trợ refresh token cookie.

Các thay đổi chính:

- Bật withCredentials: true để browser gửi HttpOnly cookie
- Tạo axios client riêng authApi cho /auth/refresh để tránh loop interceptor
- Thêm refreshPromise singleton để nhiều request 401 cùng lúc chỉ gọi refresh một lần
- Khi refresh thành công, access token mới được lưu lại
- Request gốc được retry một lần bằng token mới
- Nếu refresh thất bại, hệ thống dispatch unauthorized và redirect về login
- Trong AuthContext.jsx, hệ thống lắng nghe event auth:token-refreshed để đồng bộ lại token trong state.

Kết quả là khi access token hết hạn nhưng refresh cookie còn hợp lệ, người dùng không bị bắt đăng nhập lại ngay. Điều này cải thiện rõ rệt trải nghiệm sử dụng các màn hình cần mở lâu.

6. Chuẩn hóa exception handling và frontend error handling

6.1. Bổ sung exception handler cho auth

Backend được bổ sung nhiều exception handler cụ thể hơn cho refresh token và trạng thái user, ví dụ:

- Invalid refresh token
- Expired refresh token
- Revoked refresh token
- Missing refresh token
- Invalid reset token
- User inactive
- User not verified

Việc tách riêng các loại lỗi giúp Frontend có thể xử lý chính xác hơn, thay vì tất cả đều rơi vào một thông báo chung.

6.2. Chuẩn hóa cách đọc lỗi ở Frontend

Trước đó, một số page đọc lỗi theo `response.data.detail`, trong khi Backend chủ yếu trả response theo format có field `message`. Điều này khiến nhiều lỗi không hiển thị đúng, ví dụ login bằng tài khoản inactive có thể chỉ hiện thông báo chung chung.

Trong tuần này, em đã bổ sung các helper chung trong `api.js`:

- `getApiErrorData(error)`
- `getApiErrorMessage(error, fallback)`
- `getApiFieldErrors(error)`

Các helper này ưu tiên đọc:

- `response.data.message`
- `response.data.errors`
- `response.data.detail`

Sau đó, em thay thế các đoạn bắt lỗi rời rạc trong nhiều page như:

- Login
- ResetPassword
- VerifyEmail
- Profile
- UserManagement
- CameraManagement
- AuditLogs
- ViolationHistory
- LiveMonitoring

Kết quả là toast và form error hiển thị đồng bộ hơn với format response của Backend. Các lỗi như tài khoản bị khóa, chưa xác thực email, sai current password hoặc token hết hạn đều có thông báo rõ ràng hơn.

7. Chuẩn hóa cấu hình và tài liệu

7.1. Chuẩn hóa `.env.example`

Trong tuần này, em đã dọn lại `.env.example` và `.env.local` theo từng nhóm cấu hình rõ ràng:

- Database
- Initial Admin Account

- Authentication / JWT
- AI Inference
- Violation Detection / Deduplication
- ByteTrack Tracker
- Camera / RTSP
- Cloudinary
- Email
- Frontend / CORS

7.2. Cập nhật README

README cũng được mở rộng để mô tả rõ hơn cấu trúc hệ thống, cấu hình camera, demo video, data semantics, access control và demo runbook.

Các nội dung được bổ sung bao gồm:

- Cameras nên được quản lý qua Camera Management
- CAM_* trong .env chỉ là fallback khi DB chưa có camera active
- Webcam dùng source_type=webcam
- RTSP phone camera dùng source_type=rtsp
- Demo video dùng source_type=video_file
- Demo violation được lưu với is_demo=true
- Analytics và report mặc định loại trừ demo data
- Unverified user chỉ nên truy cập Profile và verify flow
- Admin-only gồm User Management, Camera Management và Audit Logs
- Demo runbook gồm các bước login, tạo camera, test connection, live monitoring, tạo violation, review, xem audit log và analytics

Việc bổ sung README giúp hệ thống dễ cài đặt, dễ demo và dễ giải thích hơn khi báo cáo.

8. Kết quả đạt được trong tuần 13

Sau khi hoàn thành các công việc trong tuần thứ mười ba, các kết quả chính đạt được bao gồm:

- Xác định và xử lý nguyên nhân RTSP camera điện thoại bị thấp FPS
- Bổ sung filter theo camera trong Violation History và export

- Hiển thị metadata camera trong modal ảnh vi phạm
- Cải thiện UX Camera Management khi enable/disable/delete camera
- Rà soát lại access control theo CurrentUser, ActiveUser và VerifiedUser
- Chặn user chưa verify truy cập các chức năng nghiệp vụ
- Sửa lỗi nhập sai current password làm mất token
- Chuẩn hóa xử lý token hết hạn trên Frontend
- Bổ sung refresh token backend với HttpOnly cookie
- Bổ sung refresh token rotation
- Bổ sung frontend interceptor tự refresh access token khi hết hạn
- Chuẩn hóa datetime/timestamp bằng helper riêng
- Bổ sung đầy đủ exception handler cho refresh token
- Chuẩn hóa cách đọc lỗi API trên Frontend
- Dọn lại .env.example và README

Những kết quả này giúp hệ thống ổn định hơn, trải nghiệm người dùng tốt hơn và giảm nguy cơ lỗi khi demo hoặc vận hành liên tục.

9. Khó khăn gặp phải và cách khắc phục

Trong tuần thứ mười ba, các khó khăn chủ yếu không nằm ở việc thêm chức năng mới mà nằm ở việc tích hợp, ổn định và làm rõ hành vi của hệ thống khi chạy thực tế. Các vấn đề phát sinh liên quan đến camera RTSP, token hết hạn, refresh token, datetime trong SQLite và đồng bộ lỗi giữa Backend và Frontend.

9.1. Vấn đề RTSP camera bị FPS thấp

Thách thức:

Khi xem trực tiếp từ IP Webcam, luồng video khá mượt, nhưng khi đưa vào Live Monitoring thì Capture FPS chỉ đạt khoảng 6-7 FPS. Trong khi đó, AI FPS vẫn đạt khoảng 19-21 FPS, dễ gây nhầm rằng hệ thống bị nghẽn ở mô hình AI.

Giải pháp:

Em phân tích telemetry và xác định bottleneck nằm ở capture loop. Sau đó, em viết script test độc lập để so sánh tốc độ đọc OpenCV khi không skip frame và khi grab skip 5 frame. Kết quả cho thấy không grab skip đạt khoảng 31.3 FPS, trong khi grab skip 5 frame chỉ còn khoảng 5.3 FPS. Vì vậy, em loại bỏ cơ chế grab(5) khỏi pipeline.

Kết quả:

Capture FPS tăng lại khoảng 30 FPS. Độ trễ còn khoảng 1 giây được xác định là độ trễ tự nhiên của pipeline RTSP/H264, OpenCV/FFmpeg, AI inference, encode MJPEG và browser render.

9.2. Vấn đề token hết hạn làm gián đoạn phiên làm việc

Thách thức:

Trước khi bổ sung refresh token, khi access token hết hạn, Frontend xử lý chưa rõ ràng. Người dùng có thể chỉ thấy chức năng không hoạt động hoặc bị redirect mà không có thông báo dễ hiểu.

Giải pháp:

Em bổ sung refresh token lưu trong HttpOnly cookie và cơ chế tự động refresh access token ở Frontend. Khi access token hết hạn, interceptor sẽ gọi /auth/refresh, nhận token mới và retry request cũ. Nếu refresh thất bại, hệ thống mới logout người dùng. Bên cạnh đó cũng cải thiện cách xử lý token hết hạn rõ ràng và đồng bộ hơn, thay vì để từng page tự xử lý lỗi riêng lẻ.

Kết quả:

Phiên làm việc mượt hơn và không bị gián đoạn ngay khi access token hết hạn, trong khi access token vẫn giữ được thời gian sống ngắn để đảm bảo an toàn. Kể cả khi refresh token hết hạn, hệ thống cũng có thể trả về thông báo lỗi rõ ràng hơn cho người dùng.

9.3. Vấn đề datetime naive và aware trong SQLite

Thách thức:

SQLite thường trả datetime dạng naive, trong khi code trước đó có lúc so sánh với aware datetime. Điều này có thể gây crash khi xử lý token expiry hoặc reset token.

Giải pháp:

Em tạo helper time.py và thống nhất quy ước UTC naive cho SQLite, riêng JWT exp dùng aware UTC.

Kết quả:

Các thao tác liên quan đến thời gian như refresh token, reset password, alert timestamp, camera checked time và filename export trở nên nhất quán hơn.

10. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi đã ổn định các phần quan trọng trong tuần thứ mười ba, tuần tiếp theo sẽ tiếp tục tập trung vào hoàn thiện sản phẩm cuối cùng, kiểm thử toàn diện và chuẩn bị báo cáo/bảo vệ đồ án.

10.1. Kiểm thử toàn bộ hệ thống

Trong tuần tiếp theo, em sẽ kiểm tra lại toàn bộ các luồng chính của hệ thống, bao gồm:

- Đăng nhập, refresh token và logout
- Email verification và route protection
- Camera Management
- Audit Logs
- Live Monitoring
- Switch camera
- Force stop camera
- Demo Mode
- Violation Review Workflow
- Violation History filter/export
- Analytics/report
- AI Feedback Dataset export

Mục tiêu là đảm bảo các chức năng mới không gây lỗi cho các module cũ và toàn bộ hệ thống hoạt động ổn định khi kết hợp với nhau.

10.2. Hoàn thiện migration và dữ liệu local

Nếu database local đã có dữ liệu cũ, em sẽ tiếp tục rà soát các bảng SQL và MongoDB để đảm bảo schema hiện tại đầy đủ các field mới. Ngoài ra, em có thể cân nhắc chuyển dần từ SQLite sang PostgreSQL để hệ thống phù hợp hơn với môi trường triển khai lâu dài, đặc biệt ở các vấn đề như xử lý đồng thời và lưu trữ kiểu dữ liệu thời gian. Tuy nhiên, việc chuyển đổi này sẽ cần được kiểm thử kỹ để tránh ảnh hưởng đến các chức năng hiện có.

Các phần cần kiểm tra gồm:

- RefreshToken
- Camera
- AuditLog
- User timestamp
- Violation camera metadata
- Violation is_demo
- Review fields